



**Department of Electrical, Computer
& Biomedical Engineering**
Faculty of Engineering & Architectural Science

Programs: Electrical Engineering

Course Title:	ELE632
Course Number:	Signal and Systems II
Semester/Year (e.g.F2016)	Winter/2026

Instructor:	Dr. Kandasamy Illanko
Teaching Assistant (TA)	Kevin Lowell Pham - kevin.lowell.pham@torontomu.ca

Assignment/Lab Number:	4
Assignment/Lab Title:	Discrete-Time Fourier Transform

Submission Date	Mar 22, 2026
Due Date	Mar 22, 2026

Student Last Name	Student First Name	Student ID	Section	Signature*
Abbas	Syed Maisam	501103255	05	

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

Table of Contents

Introduction.....	3
Preperation.....	3
PART A: Discrete-Time Fourier Transform (DTFT).....	4
Question 1.....	5
Question 2.....	5
Question 3.....	7
PART B: Time Convolution.....	8
Question 1.....	8
Question 2.....	9
Question 3.....	10
Question 4.....	11
Question 5.....	12
Question 6.....	12
PART C: FIR Filter Design by Frequency Sampling.....	13
Question 1.....	13
Question 2.....	13
Question 3.....	14
Question 4.....	14
Question 5.....	15
Conclusion.....	15
References.....	15
Appendix.....	16
Part A.....	16
Part B.....	17
Part C.....	19

INTRODUCTION

In this lab, the Discrete-Time Fourier transform (DTFT) will be covered. Students will discover how to compute a signal's DTFT using the FFT and look at the DTFT's time convolution property. Additionally, you will create a FIR high-pass filter and examine the distinction between a realizable filter and an ideal filter.

Previously, in Lab 1, recursive methods were used to determine the physical attributes (such as transformations, energy, and power) of discrete signals using Python algorithms and data structures. In lab 2, the basic concepts of discrete-time signal processing to analyze mathematical models and digital implementations to calculate the unit impulse response, zero-input response, zero-state response, and total response for linear time-invariant discrete-time (LTID) systems. Lab 3 allowed students to have an opportunity to learn about the Discrete-Time Fourier Series (DTFS) through the creation of a program in Python that implements the polyphase decomposition of the DTFS, as well as learning about the properties associated with the DTFS.

Throughout these experiments, the following student number will be computed as per the given sequence of variables.

$$\begin{aligned} \text{Student ID} &= 501103255 & [0] \\ [A = 5, B = 0, C = 1, D = 1, E = 0, F = 3, G = 2, H = 5, I = 5] \end{aligned}$$

Equation 0: Assigned Student Number Sequence

PREPERATION

The Discrete-time Fourier transform for a signal $x[n]$ is defined as follows, where, the signal $x[n]$ is discrete, but its Discrete Time Fourier Transform (DTFT) is a continuous signal in the frequency domain

$$X(\Omega) = \sum_{n=-\infty}^{\infty} x(n)e^{-j\Omega n} \quad [1]$$

Equation 1: Discrete-Time Fourier Transform (DTFT)

$$x[n] = \frac{1}{2\pi} \int_{2\pi} X(\Omega)e^{jn\Omega} d\Omega \quad [2]$$

Equation 2: Inverse Discrete-Time Fourier Transform (IDTFT)

The formula for the DTFT is provided in Equation [1] above and is comparable with Equation (9.30) from the textbook. Similarly, the formula for the IDTFT is provided in Equation [3] above and is comparable with Equation (9.29) from the textbook provided below.

$$x[n] = \frac{1}{2\pi} \int_{2\pi} X(\Omega) e^{jn\Omega} d\Omega \quad (9.29)$$

$$X(\Omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\Omega n} \quad (9.30)$$

Figure 1: Linear Systems and Signals Formulas for DTFT and IDTFT (*B. P. Lathi*)^[1]

Using the NumPy's FFT (Fast Fourier Transform) function, the Discrete-Time Fourier Series (DTFS) was computed. Then, following the computation given below, the Magnitude and Phase Spectra plots were generated. Note the range of n had to be changed from -10 to 10 to 1 to 20 to accommodate the magnitude and phase spectra plot generation.

```
x_vals = x(n)
X = np.fft.fft(x_vals)
```

Image 0: Fast Fourier Transform (FFT) algorithm

PART A: Discrete-Time Fourier Transform (DTFT)

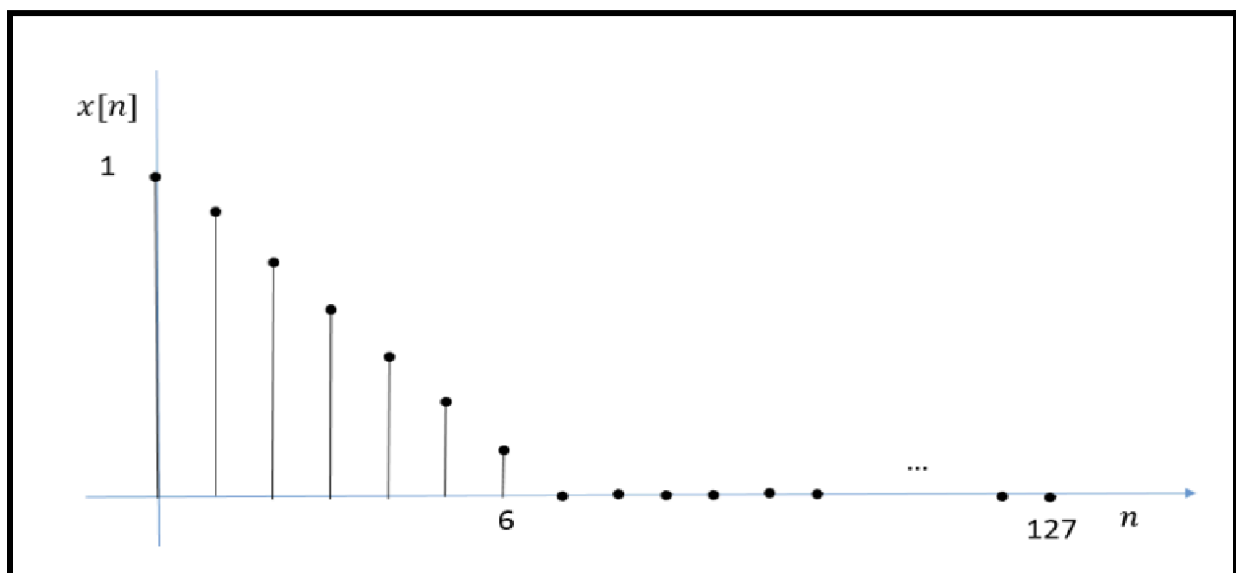


Image 1: Input Signal ($x[n]$) with Length $N_0 = 128$

Question 1:

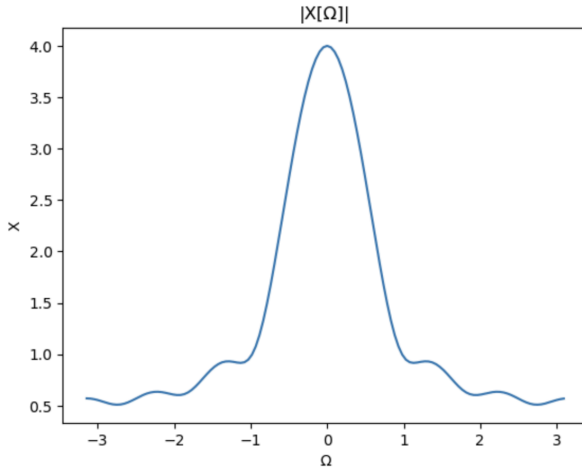


Figure 2: Magnitude Spectra ($|X(\Omega)|$)

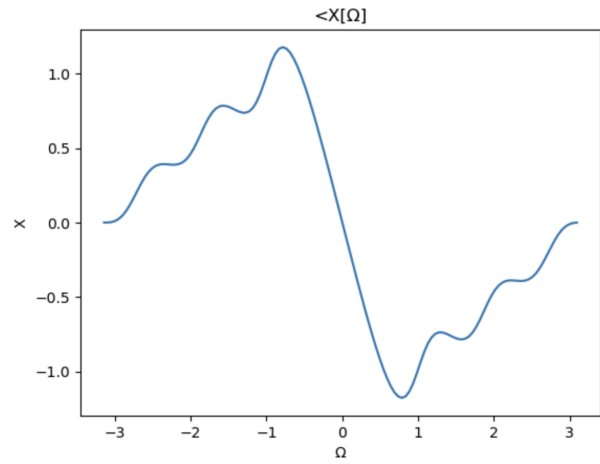


Figure 3: Phase Spectra ($\angle X(\Omega)$)

NOTE: The above plots have been shifted to be centred at $\Omega = 0$ (zero frequency component)

Question 2:

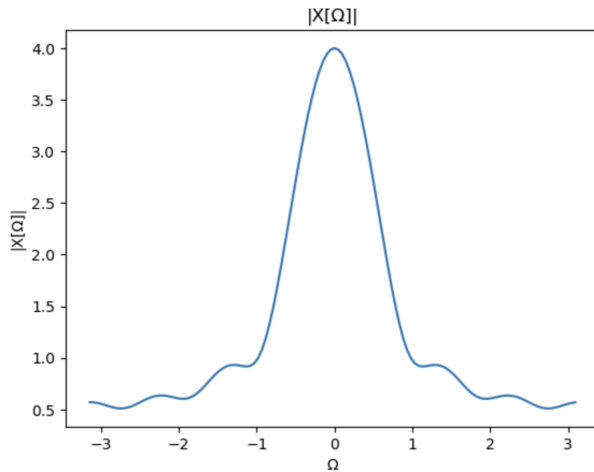


Figure 4: Magnitude Spectra ($|X(\Omega)|$)

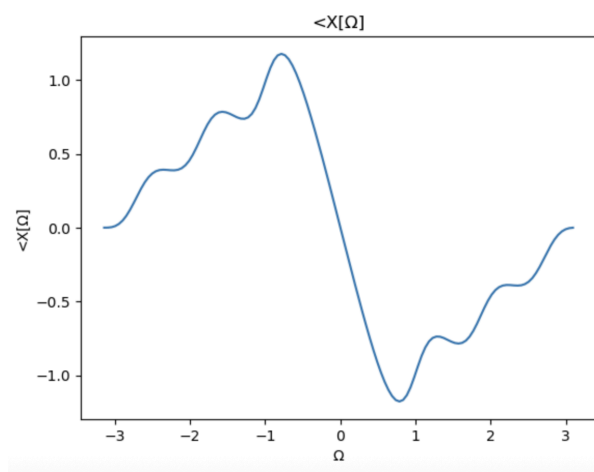


Figure 5: Phase Spectra ($\angle X(\Omega)$)

The spectrum for the DTFT $\{x[n]\}$ is composed of the magnitude and phase spectra. The magnitude ($|X(\Omega)|$) and phase ($\angle X(\Omega)$) are computed using both the *fft* command from Python's SciPy library (as found in Image 0) and theoretically as derived on Page 6, below (referencing Equation 9.30 from the textbook, which is provided in Figure 0).

$$X(\Omega) = 1 + \frac{6}{7}e^{-j\Omega} + \frac{5}{7}e^{-j2\Omega} + \frac{4}{7}e^{-j3\Omega} + \frac{3}{7}e^{-j4\Omega} + \frac{2}{7}e^{-j5\Omega} + \frac{1}{7}e^{-j6\Omega}$$

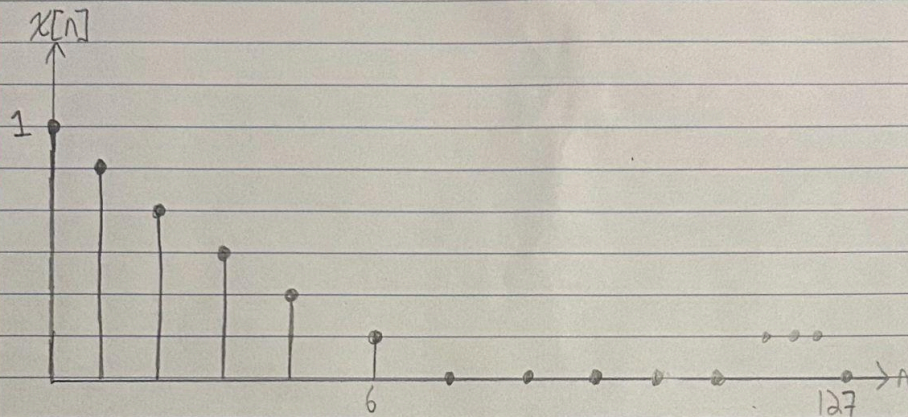
Therefore, by comparing the plots for these two (2) methods of computation, it can be concluded that Figures 2 & 4 ($|X(\Omega)|$) and Figures 3 & 5 ($\angle X(\Omega)$) are consistent and identical.

Page:

①

Lab 4: Discrete-Time Fourier Transform (DTFT)

Part A: Discrete-Time Fourier Transform (DTFT)



2) The DTFT of $x[n]$, given by $X(e^{j\omega})$, can be computed as follows:

$$\hookrightarrow X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

• Where $x[n] = \left(-\frac{1}{7}n + 1 \right) (u[n] - u[n-6])$

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} \left(-\frac{1}{7}n + 1 \right) (u[n] - u[n-6]) e^{-j\omega n}$$

$$= \sum_{n=0}^6 \left(-\frac{1}{7}n + 1 \right) e^{-j\omega n}$$

$$= -\frac{1}{7} \sum_{n=0}^6 n e^{-j\omega n} + \sum_{n=0}^6 e^{-j\omega n}$$

$$= (0+1) + \left(\frac{-e^{-j\omega} + e^{-j2\omega}}{7} \right) + \left(\frac{-2e^{-j2\omega} + e^{-j3\omega}}{7} \right) + \left(\frac{-3e^{-j3\omega} + e^{-j4\omega}}{7} \right) + \dots$$

$$\dots + \left(\frac{-5e^{-j5\omega} + e^{-j6\omega}}{7} \right) + \left(\frac{-6e^{-j6\omega} + e^{-j7\omega}}{7} \right)$$

$$= 1 + \frac{6}{7} e^{-j\omega} + \frac{5}{7} e^{-j2\omega} + \frac{4}{7} e^{-j3\omega} + \frac{3}{7} e^{-j4\omega} + \frac{2}{7} e^{-j5\omega} + \frac{1}{7} e^{-j6\omega}$$

Question 3:

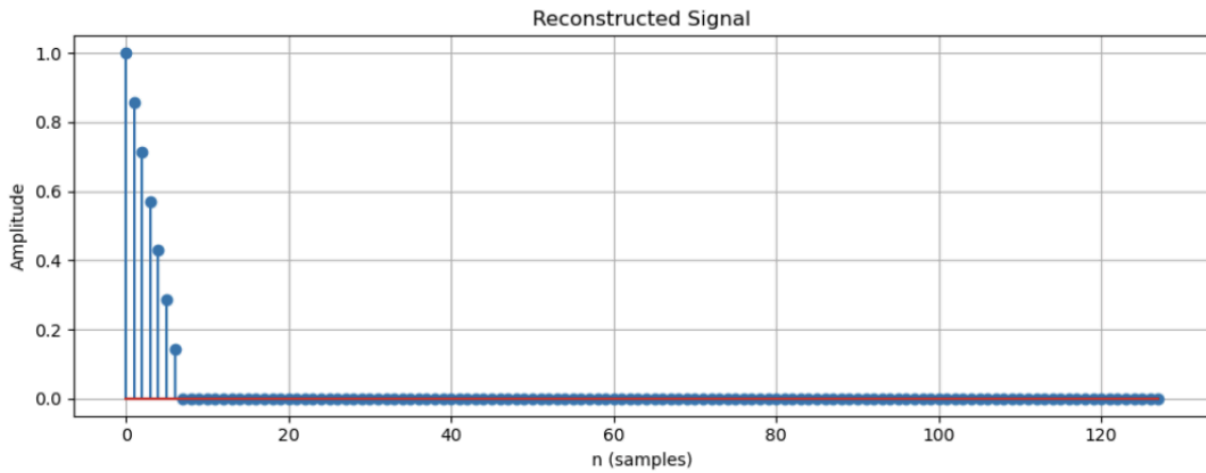


Figure 6: Time-Domain Plot for $x[n]$ using the IDTFT Spectrum of $X(\Omega)$

By referencing the above plot in Figure 6, it is clear that the Inverse Discrete-Time Fourier Transform (IDTFT) for the spectrum of $X(\Omega)$ is identical to the input signal ($x[n]$) with length $N_0 = 128$ as provided in Image 1.

Thus, the following assumption holds true as referenced from the derived equations from the textbook and Figure 1:

$$X(\Omega) = \text{DTFT}\{x[n]\} \quad \text{and} \quad x[n] = \text{IDTFT}\{X(\Omega)\}$$

Therefore, Figure 6 is identical to the original signal ($x[n]$ in Image 1). This proves that it is possible to take the Fourier Transform of a Discrete-Time signal and recover it by taking the inverse Fourier Transform.

PART B: Time Convolution

In this section, the time convolution property of the Discrete-Time Fourier transform is examined for an input signal $x[n]$. Where $x[n] = \sin(\frac{2\pi n}{(S+1)})(u[n] - u[n - (S + 1)])$ and S is given by the total, single-digit summation of the corresponding student number. Referencing Equation 0 (on page 3), the computation of S is calculated below:

$$S = A + B + C + D + E + D + F + H + I$$

$$S = 5 + 0 + 1 + 1 + 0 + 3 + 2 + 5 + 5$$

$$S = 22$$

$$S = 2 + 2$$

$$S = 4$$

```
omega = np.linspace(-np.pi, np.pi, 1001)

W_omega = np.exp(-1j)**np.dot((np.arange(x.size).reshape(x.size,1),
                                omega.reshape(1,1001)))

X = np.dot(x, W_omega)
```

Image 2: Code to Compute the Discrete Time Fourier Transform (DTFT) of a signal $x[n]$

Question 1: DTFT of the signal $x[n] = \sin(\frac{2\pi n}{5})(u[n] - u[n - 5])$

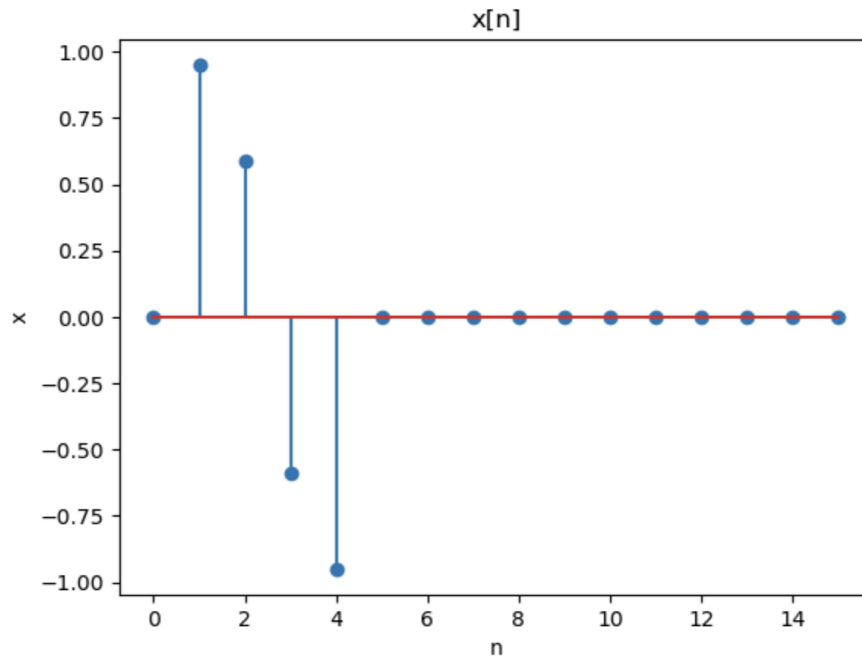


Figure 7: Plot of the Signal for $x[n]$

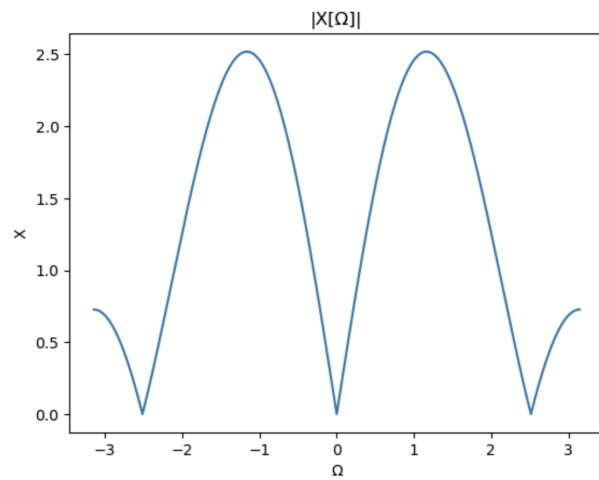


Figure 8: Magnitude Spectra ($|X(\Omega)|$)

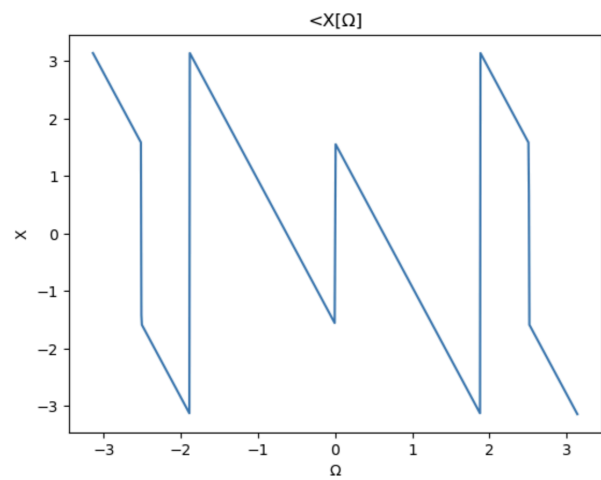


Figure 9: Phase Spectra ($\angle X(\Omega)$)

Question 2:

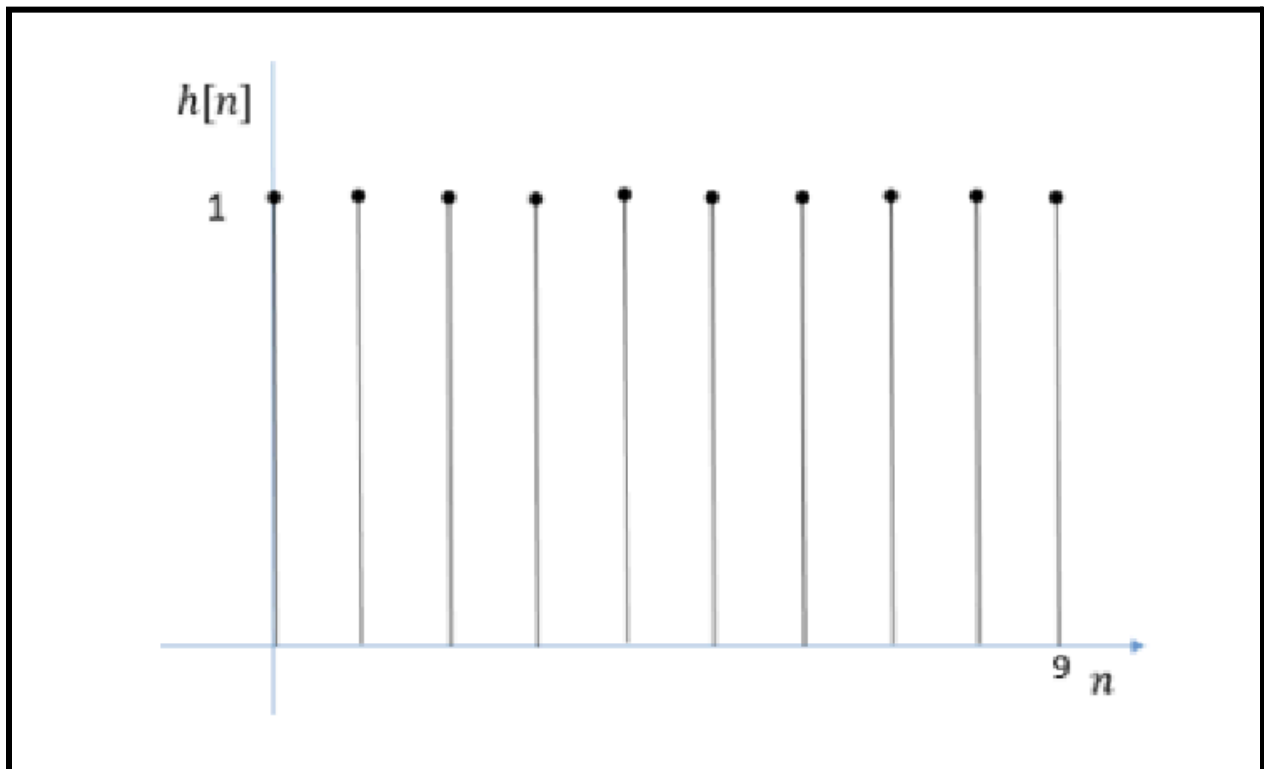


Image 3: Sketch for the Signal of $h[n]$

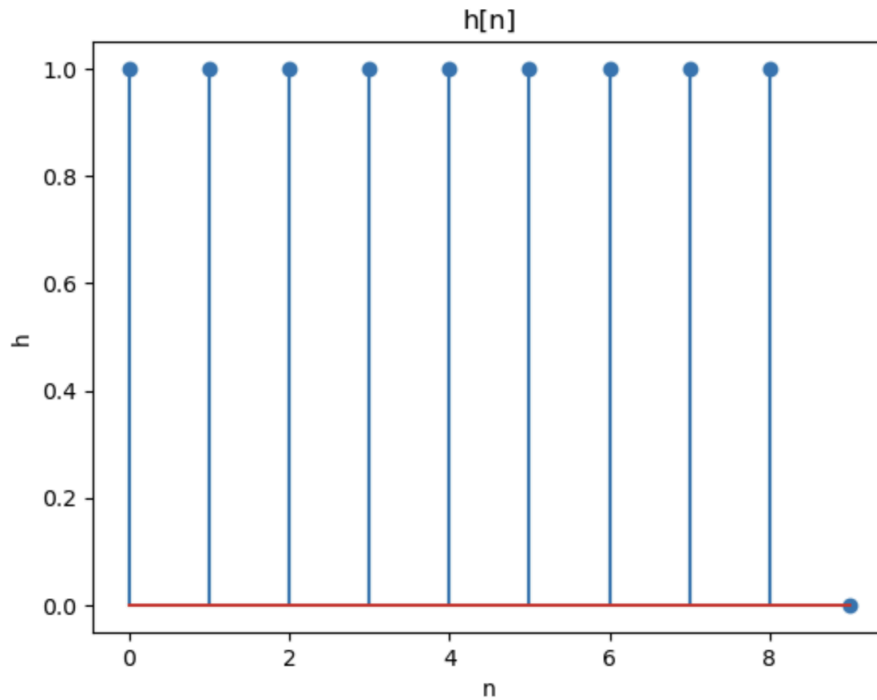


Figure 10: Simulation-Based Implementation of the Signal for $h[n]$

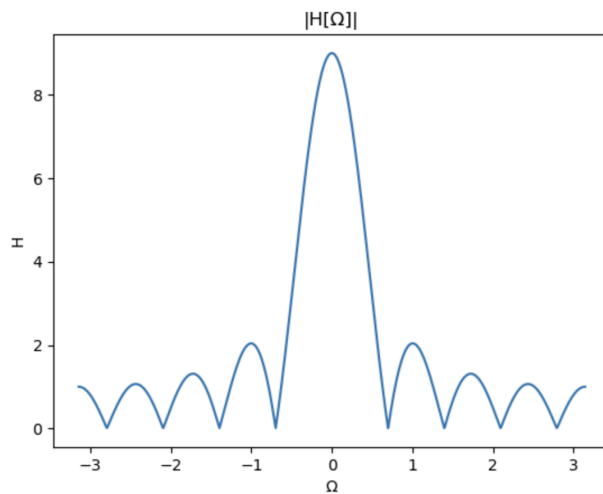


Figure 11: Magnitude Spectra ($|H(\Omega)|$)

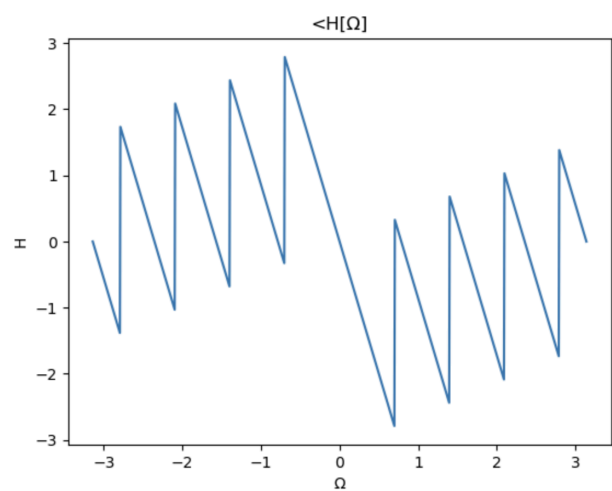


Figure 12: Phase Spectra ($\angle H(\Omega)$)

Question 3:

The time convolution of the input signals is given by the following property:

$$x[n] * h[n] = X(\Omega)H(\Omega).$$

The Discrete Time Fourier Transform (DTFT) magnitude and phase spectra for the output $y[n] = Y(\Omega)$ are provided in the plots below (Figures 13 & 14).

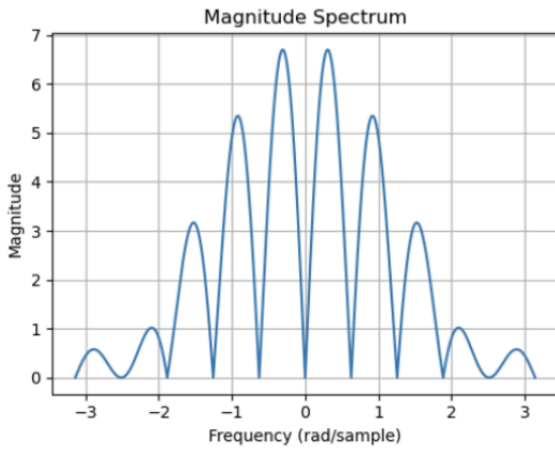


Figure 13: Magnitude Spectra ($|Y(\Omega)|$)

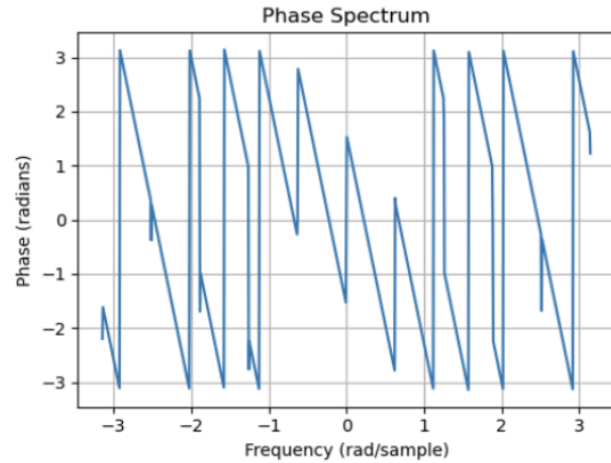


Figure 14: Phase Spectra ($\angle Y(\Omega)$)

Question 4:

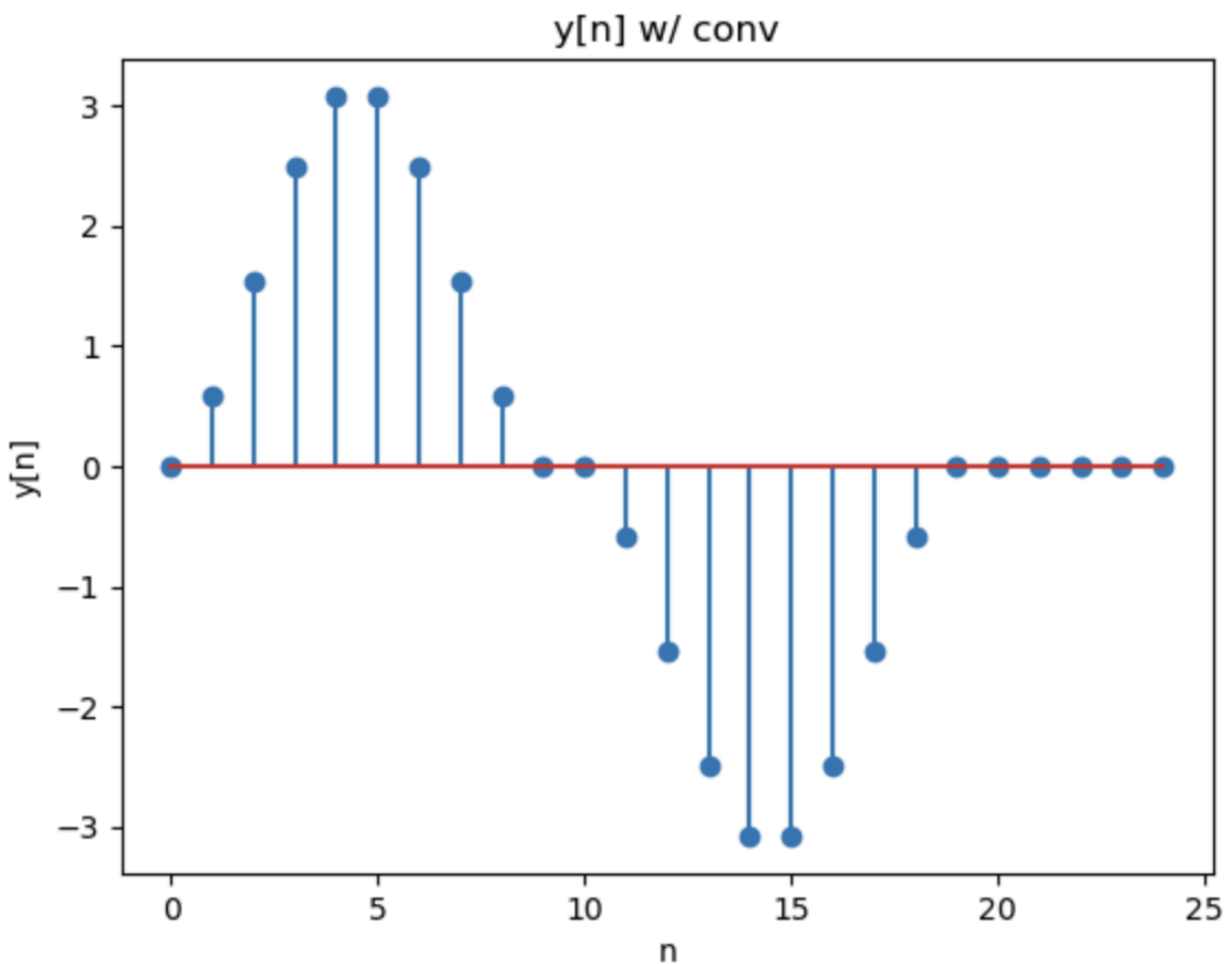


Figure 15: Output ($y[n]$) computed using the *convolve* command from the *SciPy* library

Question 5:

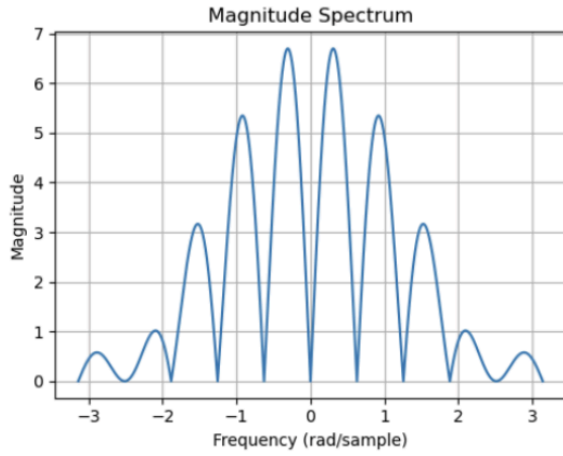


Figure 16: Magnitude Spectra ($|Y(\Omega)|$)

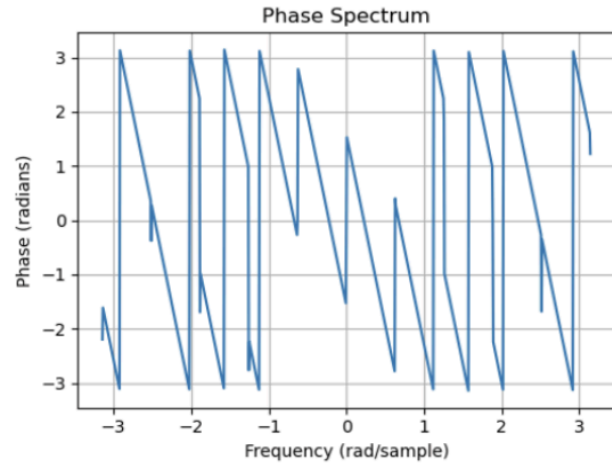


Figure 17: Phase Spectra ($\angle Y(\Omega)$)

Question 6:

$y[n] = x[n] * h[n] \quad (9.62)$ According to Eq. (9.58a) it follows that $Y(\Omega) = X(\Omega)H(\Omega) \quad (9.63)$ then $x_1[n] \iff X_1(\Omega) \quad \text{and} \quad x_2[n] \iff X_2(\Omega)$ and $x_1[n] * x_2[n] \iff X_1(\Omega)X_2(\Omega) \quad (9.58a)$ where $x_1[n] * x_2[n] = \sum_{m=-\infty}^{\infty} x_1[m]x_2[n-m]$ For two continuous, periodic signals, we define the periodic convolution, denoted by symbol $\mathbb{P}$ as^[1] $X_1(\Omega) \mathbb{P} X_2(\Omega) = \frac{1}{2\pi} \int_{2\pi} X_1(u)X_2(\Omega-u) du$	$ Y(\Omega) = X(\Omega) H(\Omega) \quad (9.64)$ and $\angle Y(\Omega) = \angle X(\Omega) + \angle H(\Omega) \quad (9.65)$ which shows that the system response to $e^{j\Omega n}$ is $H(\Omega)e^{j\Omega n}$, and $x[n] = \frac{1}{2\pi} \int_{2\pi} X(\Omega)e^{j\Omega n} d\Omega$ which shows $x[n]$ as a sum of everlasting exponential components. Invoking the linearity property, we obtain, $y[n] = \frac{1}{2\pi} \int_{2\pi} X(\Omega)H(\Omega)e^{j\Omega n} d\Omega$ which gives $y[n]$ as a sum of responses to all input components and is equivalent to Eq. (9.63). Thus, $X(\Omega)$ is the input spectrum and $Y(\Omega)$ is the output spectrum, given by $X(\Omega)H(\Omega)$.
--	--

Image 4: Time-and-Frequency Convolution Property of DTFT (B. P. Lathi)^[1]

Yes, the results are identical in questions 3 and 5. This can be referenced by Figures 13 & 16 ($|Y(\Omega)|$) and Figures 15 & 17 ($\angle Y(\Omega)$). This is because for the Fourier transform, the convolution of two functions in the time domain is identical to the product in the frequency domain, and vice versa.

Moreover, this property of the Discrete-Time Fourier Transform is further explain in detail in the textbook as referencing the derived formulas (Equations 9.62 & 9.63) in Image 4.

PART C: FIR Filter Design by Frequency Sampling

Question 1:

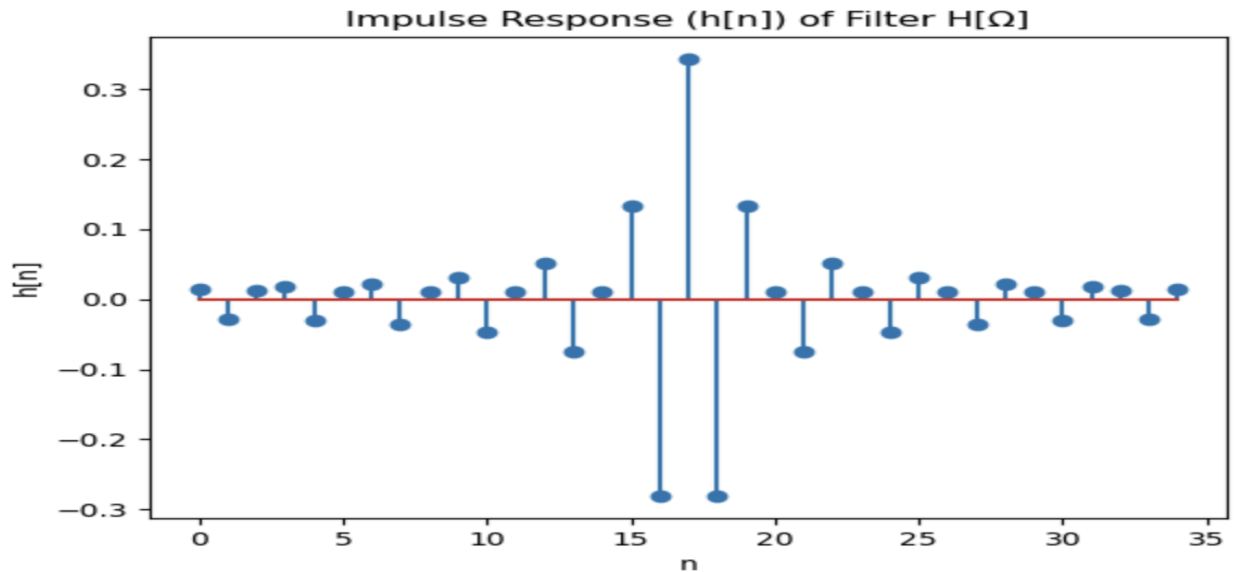


Figure 18: Impulse Response ($h[n]$) of the High-Pass FIR (Finite Impulse Response) Filter

NOTE: The Cutoff Frequency is given by $\Omega_0 = \frac{2\pi}{3}$ and the Filter Length is $N_0 = 35$.

Question 2:

```
w, H = signal.freqz(h, 1, worN = np.arange(0, 2*np.pi, 2*np.pi/1001))
```

Image 5: Python Command *freqz* to Compute the Frequency Response of the Filter from $h[n]$

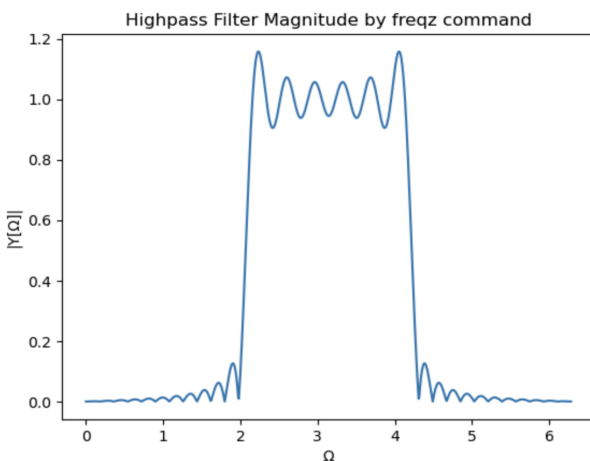


Figure 19: Magnitude Spectra ($|H(\Omega)|$)

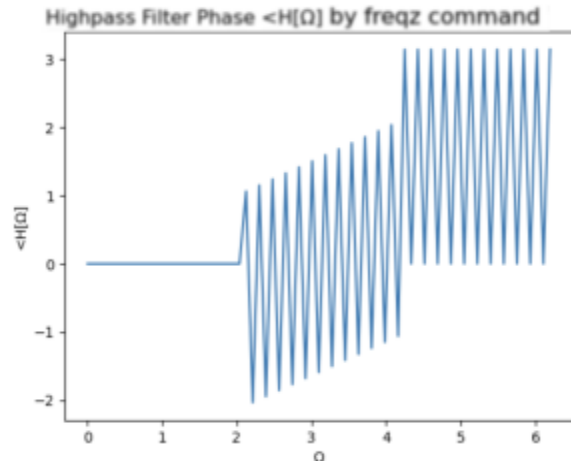


Figure 20: Phase Spectra ($\angle H(\Omega)$)

Question 3:

The result in part 2 is different from the ideal filter initially started with based on the allowed frequencies. The ideal filter has a strict cutoff frequency (with $\Omega_0 = \frac{2\pi}{3}$), with a narrow bandwidth.

This is because the impulse response of an ideal highpass filter is noncausal and unrealizable. Whereas the part 2 plot has ripples beyond the desired cutoff frequency in other sections and even in that part.

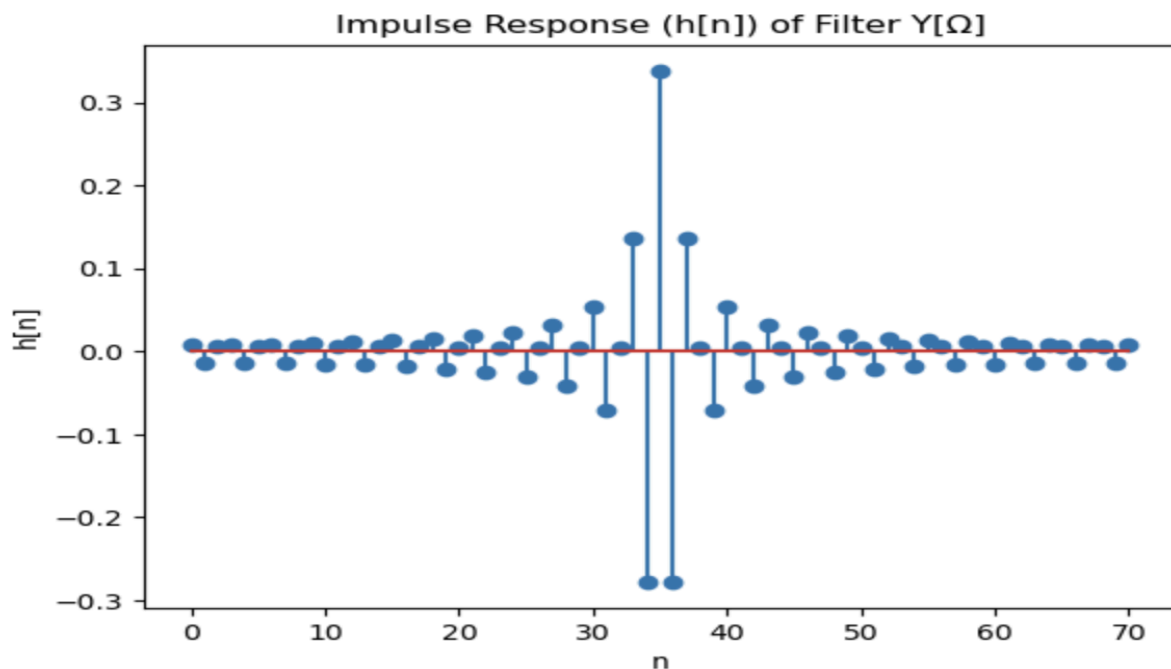
Question 4:

Figure 21: Plot of $h[n]$ with an increased $N_0 = 71$

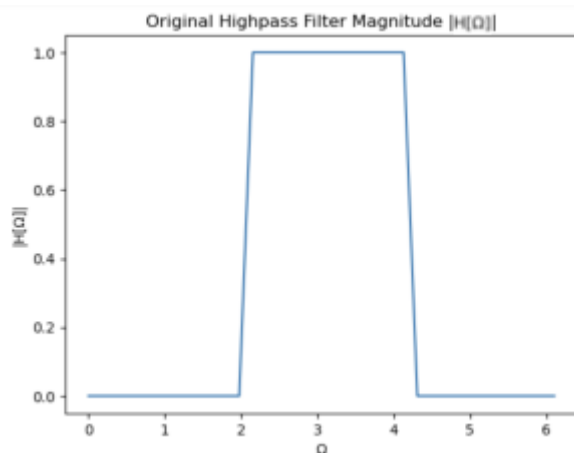


Figure 22: Magnitude Spectra ($|H(\Omega)|$)

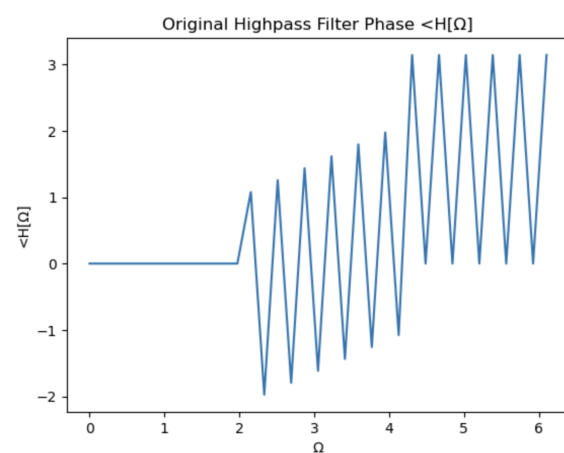


Figure 23: Phase Spectra ($\angle H(\Omega)$)

Question 5:

This increase in accuracy produces increased transitions and higher frequencies as given in the magnitude and phase plots, respectively. The magnitude and phase spectra of the original impulse response $h[n]$ given in Figures 22 & 23 and the increased filter length magnitude and phase spectra given in Figures 19 & 20. Due to the existence of more points and a more accurate representation, as more peaks are plotted, closer together, meaning each one will conform closer and closer to the original signal.

Therefore, increasing N increases the resolution as a more accurate representation of the impulse response as noticed in Figure 18 ($N_0 = 35$) and Figure 21 ($N_0 = 71$).

CONCLUSION

Using both theoretical derivation and Python-based simulation, this lab effectively investigated the basic ideas and real-world applications of the Discrete-Time Fourier Transform (DTFT). We confirmed the consistency between numerical FFT techniques and analytical "by hand" expansions by utilizing the exercises in Part A to confirm that a discrete signal can be successfully transformed into the frequency domain and reliably recovered using the Inverse DTFT (IDTFT).

The substantial computational benefit of the frequency domain was illustrated by the study of Time Convolution in Part B. We verified the convolution property, a fundamental component of linear system analysis, by verifying that $y[n] = x[n] * h[n]$ is equal to $Y(\Omega) = X(\Omega)H(\Omega)$. This comparison demonstrated how point-wise multiplication in the frequency domain can be used to simplify complex time-domain interactions.

Lastly, Part C's creation of a High-Pass FIR Filter brought to light the crucial differences between ideal mathematical models and practical digital filters. It is technically impossible to create ideal filters since they are noncausal and need an indefinite impulse response. Due to the Gibbs Phenomenon, realizable filters, which are produced by truncating the impulse response to $N_0 = 35$, introduce distortions like ripples and a finite transition bandwidth. By decreasing the transition band and raising the resolution of the frequency response, increasing the filter length (from $N_0 = 35$ to $N_0 = 71$) greatly enhances the filter's performance and provides a much closer approach to the intended cutoff frequency $\Omega_c = \frac{2\pi}{3}$.

REFERENCES

[1] Lathi, B. P., & Green, R. A. (2018). Linear Systems and Signals (3rd ed.). Oxford University Press.

APPENDIX

PART A

Question 1:

```
import numpy as np
import matplotlib.pyplot as plt

N0 = 128
n = np.arange(N0)
ohm = (2*np.pi/128)*np.arange(-64, 64)

x = [1, 6/7, 5/7, 4/7, 3/7, 2/7, 1/7] + [0]*121
X = np.fft.fft(x)

plt.figure()
plt.plot(ohm, np.fft.fftshift(np.abs(X)))
plt.title('|X[Ω]|')
plt.xlabel('Ω')
plt.ylabel('X')

plt.figure()
plt.plot(ohm, np.fft.fftshift(np.angle(X)))
plt.title('<X[Ω]')
plt.xlabel('Ω')
plt.ylabel('X')

plt.show()
```

Question 2:

```
import numpy as np
import matplotlib.pyplot as plt

# Clear command window
from IPython.display import clear_output
clear_output(wait=True)

ohm = (2*np.pi/128)*np.arange(-64, 64)

X = 1 + (6/7)*np.exp(-1j*ohm) \
    + (5/7)*np.exp(-1j*2*ohm) \
    + (4/7)*np.exp(-1j*3*ohm) \
    + (3/7)*np.exp(-1j*4*ohm) \
    + (2/7)*np.exp(-1j*5*ohm) \
    + (1/7)*np.exp(-1j*6*ohm)

plt.figure()
plt.plot(ohm, np.abs(X))
plt.title('|X[Ω]|')
plt.xlabel('Ω')
plt.ylabel('|X[Ω]|')

plt.figure()
plt.plot(ohm, np.angle(X))
```

```
plt.title('<X[Ω]')
plt.xlabel('Ω')
plt.ylabel('<X[Ω]')

plt.show()
```

Question 3:

```
import numpy as np
import matplotlib.pyplot as plt

N0 = 128
n = np.arange(N0)
ohm = (2*np.pi/128)*np.arange(-64, 64)

X = 1 + (6/7)*np.exp(-1j*ohm) + (5/7)*np.exp(-1j*2*ohm) + (4/7)*np.exp(-1j*3*ohm) +
(3/7)*np.exp(-1j*4*ohm) \
+ (2/7)*np.exp(-1j*5*ohm) \
+ (1/7)*np.exp(-1j*6*ohm)

x = np.fft.ifft(X)

plt.figure()
plt.stem(n, x.real)
plt.title('x[n]')
plt.xlabel('n')
plt.ylabel('x')

plt.show()
```

PART B

Question 1:

```
n = np.arange(16)
u_c = lambda t: 1.0 * (t >= 0)
u = lambda n: u_c(n) * (np.mod(n, 1) == 0)
x = np.sin(2 * np.pi * n / 5) * (u(n) - u(n - 5))

omega = np.linspace(-np.pi, np.pi, 1001)
W_omega = np.exp(-1j) ** (np.outer(np.arange(len(x)), omega))
X = np.dot(x, W_omega)

plt.stem(n, x)
plt.title('x[n]')
plt.xlabel('n')
plt.ylabel('x')
plt.show()

plt.plot(omega, np.abs(X))
plt.title('|X[Ω]|')
plt.xlabel('Ω')
plt.ylabel('X')
plt.show()
```

```
plt.plot(omega, np.angle(X))
plt.title('<X[Ω]')
plt.xlabel('Ω')
plt.ylabel('X')
plt.show()
```

Question 2:

```
n = np.arange(10)
x = np.heaviside(n, 1) - np.heaviside(n-9, 1)

omega = np.linspace(-np.pi, np.pi, 1001)
W_omega = np.exp(-1j*np.outer(np.arange(len(x)), omega))
H = x @ W_omega

plt.stem(n, x)
plt.title('h[n]')
plt.xlabel('n')
plt.ylabel('h')
plt.show()

plt.plot(omega, np.abs(H))
plt.title('|H[Ω]|')
plt.xlabel('Ω')
plt.ylabel('H')
plt.show()

plt.plot(omega, np.angle(H))
plt.title('<H[Ω]')
plt.xlabel('Ω')
plt.ylabel('H')
plt.show()
```

Question 3:

```
Y = X * H

plt.plot(omega, np.abs(Y))
plt.title('|Y[Ω]|')
plt.xlabel('Ω')
plt.ylabel('Y')
plt.show()

plt.plot(omega, np.angle(Y))
plt.title('<Y[Ω]')
plt.xlabel('Ω')
plt.ylabel('Y')
plt.show()
```

Question 4:

```
n = np.arange(25)
h = np.heaviside(np.arange(10), 1)
x = np.sin(2*np.pi*n/10) * (np.heaviside(n, 1) - np.heaviside(n-10, 1))

y = np.convolve(x, h)

plt.stem(n, y[:25])
plt.title('y[n] w/ conv')
plt.xlabel('n')
plt.ylabel('y[n]')
plt.show()
```

Question 5:

```
omega = np.linspace(-np.pi, np.pi, 1001)
W_omega = np.exp(-1j*np.outer(np.arange(len(y)), omega))
Y = y @ W_omega

plt.plot(omega, np.abs(Y))
plt.title('|Y[Ω]|')
plt.xlabel('Ω')
plt.ylabel('Y')
plt.show()

plt.plot(omega, np.angle(Y))
plt.title('<Y[Ω]')
plt.xlabel('Ω')
plt.ylabel('Y')
plt.show()
```

PART C

```
# Part C1,2,4

ohm0 = 2*np.pi/3
N = 35
n = np.arange(N)
Omega = np.linspace(0, 2*np.pi*(1-1/N), N)
H_d = lambda Omega: (np.mod(Omega, 2*np.pi) > ohm0) * (np.mod(Omega, 2*np.pi) < 2*np.pi - ohm0)

H = H_d(Omega) * np.exp(-1j * Omega * ((N-1)/2))
h = np.fft.iffit(H)

plt.figure()
plt.plot(Omega, abs(H))
plt.title('Original Highpass Filter Magnitude |Y[Ω]|')
plt.xlabel('Ω')
plt.ylabel('|H[Ω]|')

plt.figure()
plt.plot(Omega, np.angle(H))
plt.title('Original Highpass Filter Phase <H[Ω]')
```

```

plt.xlabel('Ω')
plt.ylabel('<H[Ω]')

plt.figure()
plt.stem(n, h)
plt.title('Impulse Response (h[n]) of Filter H[Ω]')
plt.xlabel('n')
plt.ylabel('h[n]')

H = np.fft.fft(h, 1001)
Omega_H = np.linspace(0, 2*np.pi, 1001)
plt.figure()
plt.plot(Omega_H, abs(H))
plt.title('Highpass Filter Magnitude by freqz command')
plt.xlabel('Ω')
plt.ylabel('|Y[Ω]|')

plt.show()

ohm0 = 2*np.pi/3
N = 71
n = np.arange(N)
Omega = np.linspace(0, 2*np.pi*(1-1/N), N)
H_d = lambda Omega: (np.mod(Omega, 2*np.pi) > ohm0) * (np.mod(Omega, 2*np.pi) < 2*np.pi - ohm0)

H = H_d(Omega) * np.exp(-1j*Omega*((N-1)/2))
h = np.fft.ifft(H)

plt.figure()
plt.plot(Omega, np.abs(H))
plt.title('Original Highpass Filter')
plt.xlabel('Ω')
plt.ylabel('|Y[Ω]|')

plt.figure()
plt.plot(Omega, np.angle(H))
plt.title('Original Highpass Filter Phase <H[Ω]')
plt.xlabel('Ω')
plt.ylabel('<H[Ω]')

plt.figure()
plt.stem(n, h)
plt.title('Impulse Response (h[n]) of Filter Y[Ω]')
plt.xlabel('n')
plt.ylabel('h[n]')

plt.show()

```